A Lab Project Report on

DAILY TASK MANAGER APP USING FLUTTER

Submitted in partial fulfilment of the requirement of the lab

FLUTTER LAB

III-B. TECH. I SEM

in

Department of CSE(AI & ML)

Submitted by

CH.SRIDEVI (247R1A6618)

C.SAKSHITHA (247R1A6619)

I.VEDASRI (247R1A6629)



DEPARTMENT OF CSE(AI & ML)

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NBA, Approved by AICTE, Affiliated to JNTUH)

Kandlakoya, Medchal Road, Hyderabad-501401

2026-2027

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Approved by AICTE, Affiliated to JNTUH, Kukatpally, Hyderabad)

Kandlakoya, Medchal Road, Hyderabad-501401.

DEPARTMENT OF CSE(AI & ML)



CERTIFICATE

This is to certify that a Mini Project entitled with: “DAILY TASK MANAGER APP USING FLUTTER” is being Submitted by CH.SRIDEVI (247R1A6618), C.SAKSHITHA (247R1A6619), I.VEDASRI (247R1A6629) in partial fulfilment of the requirement for completion of the FLUTTER LAB III B.Tech I- Semester is a record of a bonafide work carried out under guidance and supervision.

Signature of Faculty **Signature of the HOD**

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Raji Reddy, Director, and Dr. S Rao Chintalapudi, Head of Department,** Department of CSE (AI & ML), CMR TECHNICAL CAMPUS, for their inspiration and valuable guidance during the entire duration. We are extremely thankful to our Assistant Professor **G M Subhani**, Department of CSE (AI & ML), for his constant guidance, encouragement, and moral support throughout the project. We express our gratitude to all staff members and friends for the help and coordination they extended in bringing this mini project to a successful completion on time.

CH.SRIDEVI (247R1A6618)
C.SAKSHITHA (247R1A6619)
I.VEDASRI (247R1A6629)

The table of contents outlines the project's structure, covering:

- Abstract
- CHAPTER 1: Introduction
    - Introduction to the Daily Task Manager (Definition, purpose, importance, features)
    - Introduction to Flutter and Dart (History, features, why Flutter is used for the project, advantages)
- CHAPTER 2: Literature Survey
    - Task Management Applications (Overview, features/limitations, comparison)
    - Graphical User Interface (GUI) for the Task Manager (Importance, examples)
- CHAPTER 3: Analysis and Design
    - System Architecture (Overview, components/modules)
    - System Design (Detailed design)
- CHAPTER 4: Implementation
    - SharedPreferences and State Management in Flutter (Introduction, use in the task manager)
    - Code (Complete code for the project)
- CHAPTER 5: Results
    - Output of the Daily Task Manager (Screenshots/examples, discussion of results)
- CHAPTER 6: Conclusion and future scope
    - Summary of the project (Recap, limitations)
    - Future scope and enhancements (Ideas for improvements)
- References

Abstract

This lab project presents the Daily Task Manager, a mobile application built with Flutter and Dart that helps a user plan and track daily work. The user can add a task with a title, an optional description and a date, edit it later, mark it as completed with a checkbox, delete it after confirmation, and filter the list into All, Pending and Completed tasks. Tasks are converted to JSON and stored on the device with the shared_preferences package, so they remain available after the app is closed and no internet connection or account is needed. The code is organised into a model, a storage service, reusable widgets and three screens, with form validation and clear feedback through dialogs and snack bars. The project demonstrates stateful widgets, navigation, forms, date picking, local persistence and automated testing in Flutter.

CHAPTER 1: Introduction

Introduction to the Daily Task Manager

1.1 Definition and purpose of a task manager: A task manager is an application that lets a person record the work they have to do, track its progress and mark it complete. The purpose of this project is to build a simple and reliable daily task manager for a mobile phone that stores everything locally, while demonstrating the main ideas of Flutter application development.

1.2 Importance of task management in daily life: Students and professionals handle assignments, lab records, exams and personal work at the same time. Writing tasks down and ticking them off reduces forgotten deadlines and gives a clear picture of what is pending. A phone based manager is always at hand, which makes it more practical than paper lists.

1.3 Features of the Daily Task Manager: The app lets the user add, edit and delete tasks, choose a task date with the Material date picker, and mark tasks complete with a checkbox so the title is shown with a strike-through. A segmented button filters the list into All, Pending and Completed. The home screen shows today's date, an empty state message when there are no tasks, a delete confirmation dialog and snack bar messages for every action. All data is saved on the device with SharedPreferences.

Introduction to Flutter and Dart

1.4 Brief history and features of Flutter and Dart: Dart is a strongly typed, object-oriented language developed by Google and announced in 2011. Flutter is Google's open source UI toolkit, first announced in 2015 and released as version 1.0 in December 2018. Everything in a Flutter interface is a widget, the framework supports hot reload, and one code base compiles to native code for Android, iOS, web and desktop.

1.5 Why Flutter is used for the project: Flutter provides ready-made Material 3 widgets such as Card, SegmentedButton, FloatingActionButton, TextFormField and showDatePicker, which cover almost every need of a task manager. Its Form and validator support, Navigator and the shared_preferences plugin make it quick to build, store and test an app of this kind.

1.6 Advantages of using Flutter for the task manager: A single code base serves several platforms, hot reload shortens the development cycle, and the same widget model is

used for small parts such as the task card and for whole screens. Flutter also includes a testing framework, so the storage logic and the screens can be checked automatically.

CHAPTER 2: Literature Survey

Task Management Applications

2.1 Overview of existing task management applications: Popular applications include Google Tasks, Microsoft To Do, Todoist and Any.do. They offer lists, due dates, reminders, sharing and synchronisation across devices, and many also provide web and desktop clients.

2.2 Features and limitations of existing applications: These applications are powerful but often need an account and an internet connection, have many features that a student does not need, and may place advanced functions behind a paid plan. Their data is stored on the provider's servers, and their code is closed, so they are not useful for learning how an app is built.

2.3 Comparison of existing applications with the proposed application: Table 2.1 compares typical task applications with this project.

Table 2.1: Comparison of task managers

Feature	Popular cloud task apps	This project
Account and internet	Usually required	Not required, works offline
Data storage	Provider servers	On the device with SharedPreferences
Interface complexity	Many features and menus	Simple, three screens
Filtering	Lists, tags and search	All, Pending and Completed
Source code	Closed	Open code written for learning
Reminders and sync	Yes	Not included (future scope)

Graphical User Interface (GUI) for the Task Manager

2.4 Importance of a GUI in a task manager: A task manager is used many times a day, so the interface must be quick and clear. Large touch targets, a visible add button, a filter at the top and visual feedback such as a strike-through for finished work let the user understand the state of their tasks at a glance.

2.5 Examples of GUI toolkits and widgets used: Mobile interfaces can be built with Android Views, SwiftUI or Flutter. This project uses the Flutter Material 3 library: Scaffold and AppBar for the page structure, SegmentedButton for filters, ListView.builder with Card for the task list, FloatingActionButton for adding tasks, TextFormField for input, showDatePicker for dates, AlertDialog for confirmation and SnackBar for messages.

CHAPTER 3: Analysis and Design

System Architecture

3.1 Overview of the system architecture: The application is organised in layers, as shown in Figure 3.1. The presentation layer contains the three screens and the reusable TaskCard widget. The model layer contains the Task class. The service layer contains

StorageService, which is the only part of the app that talks to SharedPreferences. This separation keeps the screens simple and lets the storage logic be tested on its own.

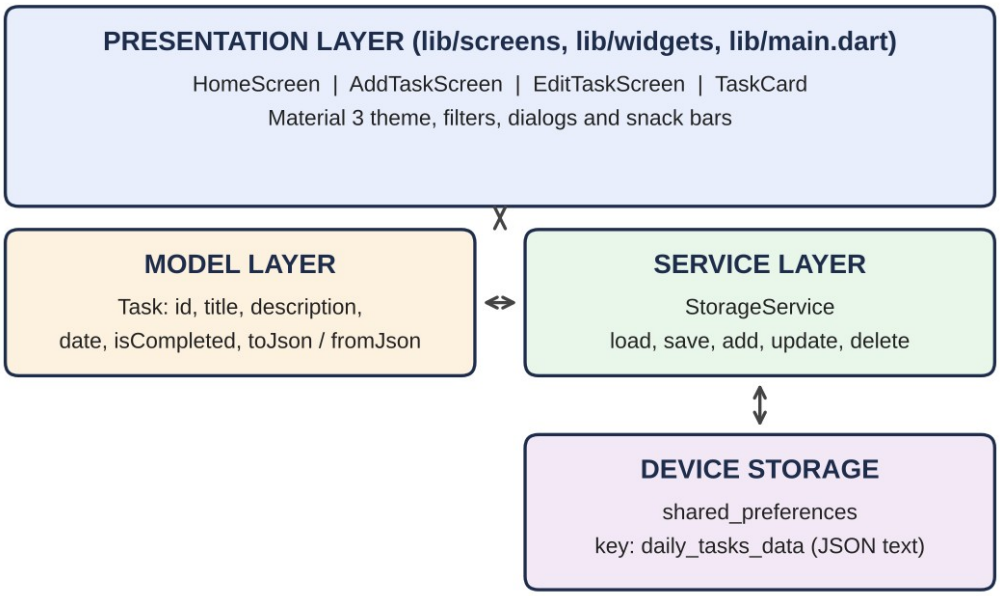


Figure 3.1: Layers of the Daily Task Manager

3.2 Components and modules used in the system: main.dart starts the app and sets the Material 3 indigo theme. models/task.dart defines the Task class with toJson and fromJson. services/storage_service.dart provides loadTasks, saveTasks, addTask, updateTask and deleteTask. widgets/task_card.dart displays one task. screens/home_screen.dart shows the list and filters, screens/add_task_screen.dart creates tasks and screens/edit_task_screen.dart changes them. The test folder contains unit_and_flow_test.dart and widget_test.dart.

System Design

3.3 Detailed design of the system: A task has five fields: id, title, description, date and isCompleted. The whole list is stored as one JSON string under the key daily_tasks_data. When the home screen opens it loads the list, shows a progress indicator and then displays the tasks. Adding a task validates the title, creates a Task whose id is the current time in milliseconds, inserts it at the top of the list and returns true to the home screen, which reloads. Editing pre-fills the form with the existing values and replaces the task with the same id. Ticking the checkbox updates the screen immediately and then saves the change. Deleting asks for confirmation, removes the task from the list and from storage and shows a snack bar. The filter is a getter that returns all tasks, only those not completed, or only those completed. Table 3.1 summarises the screens.

Table 3.1: Screens and widgets

Screen or widget	Purpose	Main widgets
HomeScreen	Show date, filter and task list; open add and edit; confirm delete	SegmentedButton, ListView.builder, FloatingActionButton, AlertDialog
AddTaskScreen	Create a task with validation	Form, TextFormField, showDatePicker, FilledButton
EditTaskScreen	Change an existing task	Same form, pre-filled from the task
TaskCard	Display one task with checkbox, date, edit and delete	Card, Checkbox, IconButton

CHAPTER 4: Implementation

SharedPreferences and State Management in Flutter

4.1 Introduction to SharedPreferences and state management: shared_preferences is a Flutter plugin that stores small key-value data permanently on the device, using SharedPreferences on Android, NSUserDefaults on iOS and local storage on the web. It saves simple types such as strings, so complex objects are first converted to JSON text with jsonEncode and read back with jsonDecode. In Flutter, a StatefulWidget keeps data that changes over time, and calling setState tells the framework to rebuild the screen with the new data.

4.2 Use of these concepts in the task manager: StorageService reads and writes the task list as a JSON string and handles errors with try and catch so that a storage problem never crashes the app. HomeScreen is a StatefulWidget that holds the task list, the loading flag and the selected filter, and uses setState after every change. Navigator.push opens the add and edit screens and returns true when something changed, which makes the home screen reload. A Form with a GlobalKey and a validator rejects an empty title, and mounted checks make sure a screen is still on display before it shows a snack bar after an asynchronous save.

Code

4.3 Complete code for the Daily Task Manager project: The source code of the application and its tests is given below. edit_task_screen.dart is the same form as add_task_screen.dart, pre-filled with the existing task and calling updateTask, so it is not repeated. task_card.dart and widget_test.dart are described in Sections 3.2 and 5.2.

Listing 4.1: lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const DailyTaskManagerApp());
}

class DailyTaskManagerApp extends StatelessWidget {
  const DailyTaskManagerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daily Task Manager',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
```

```
          brightness: Brightness.light,
        ),
        appBarTheme: const AppBarTheme(
          centerTitle: false,
          elevation: 0,
        ),
      ),
      home: const HomeScreen(),
    );
  }
}
```

Listing 4.2: lib/models/task.dart

```
// Simple Task Model representing a single daily task
class Task {
  String id;
  String title;
  String description;
  DateTime date;
  bool isCompleted;

  Task({
    required this.id,
    required this.title,
    this.description = '',
    required this.date,
    this.isCompleted = false,
  });

  // Convert a Task object into a Map (JSON) to save in SharedPreferences
  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'title': title,
      'description': description,
      'date': date.toIso8601String(),
      'isCompleted': isCompleted,
    };
  }

  // Create a Task object from a Map (JSON) loaded from SharedPreferences
  factory Task.fromJson(Map<String, dynamic> json) {
    return Task(
      id: json['id'] as String,
      title: json['title'] as String,
      description: (json['description'] as String?) ?? '',
      date: DateTime.parse(json['date'] as String),
      isCompleted: (json['isCompleted'] as bool?) ?? false,
    );
  }
}
```

Listing 4.3: lib/services/storage_service.dart

```
import 'dart:convert';
import 'package:flutter/foundation.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../models/task.dart';

// Service to manage task persistence using SharedPreferences
class StorageService {
  static const String _tasksKey = 'daily_tasks_data';

  // Load all tasks from SharedPreferences
  Future<List<Task>> loadTasks() async {
    try {
      final prefs = await SharedPreferences.getInstance();
      final String? tasksJson = prefs.getString(_tasksKey);

      if (tasksJson == null || tasksJson.isEmpty) {
        return [];
      }

      final List<dynamic> decodedList = jsonDecode(tasksJson);
      return decodedList
          .map((item) => Task.fromJson(item as Map<String, dynamic>))
          .toList();
    } catch (e) {
      debugPrint('Error loading tasks: $e');
      return [];
    }
  }

  // Save the full list of tasks to SharedPreferences
  Future<bool> saveTasks(List<Task> tasks) async {
    try {
      final prefs = await SharedPreferences.getInstance();
      final List<Map<String, dynamic>> taskListMap =
          tasks.map((task) => task.toJson()).toList();
      final String encodedJson = jsonEncode(taskListMap);
      return await prefs.setString(_tasksKey, encodedJson);
    } catch (e) {
```

```
      debugPrint('Error saving tasks: $e');
      return false;
    }
  }

  // Add a single task to storage
  Future<bool> addTask(Task task) async {
    final tasks = await loadTasks();
    tasks.insert(0, task); // Add newest task at the top
    return await saveTasks(tasks);
  }

  // Update an existing task in storage
  Future<bool> updateTask(Task updatedTask) async {
    final tasks = await loadTasks();
    final index = tasks.indexWhere((task) => task.id == updatedTask.id);
    if (index != -1) {
      tasks[index] = updatedTask;
      return await saveTasks(tasks);
    }
    return false;
  }

  // Delete a task from storage by ID
  Future<bool> deleteTask(String taskId) async {
    final tasks = await loadTasks();
    tasks.removeWhere((task) => task.id == taskId);
    return await saveTasks(tasks);
  }
}
```

Listing 4.4: lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';
import '../models/task.dart';
import '../services/storage_service.dart';
import '../widgets/task_card.dart';
import 'add_task_screen.dart';
import 'edit_task_screen.dart';

// Enum to represent the three task filter options
enum TaskFilter { all, pending, completed }

// Main Home Screen of Daily Task Manager
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final StorageService _storageService = StorageService();

  List<Task> _tasks = [];
  bool _isLoading = true;
  TaskFilter _selectedFilter = TaskFilter.all;

  @override
  void initState() {
    super.initState();
    _loadTasks();
  }

  // Load saved tasks from SharedPreferences
  Future<void> _loadTasks() async {
    setState(() {
      _isLoading = true;
    });

    final loadedTasks = await _storageService.loadTasks();

    setState(() {
      _tasks = loadedTasks;
      _isLoading = false;
    });
  }

  // Filter tasks based on current selection: All, Pending, or Completed
  List<Task> get _filteredTasks {
    switch (_selectedFilter) {
      case TaskFilter.pending:
        return _tasks.where((task) => !task.isCompleted).toList();
      case TaskFilter.completed:
        return _tasks.where((task) => task.isCompleted).toList();
      case TaskFilter.all:
        return _tasks;
    }
  }

  // Format today's date for the header: "Monday, 28 Sep 2026"
  String _formatTodayDate(DateTime date) {
    const weekdays = [
```

```
    'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'
  ];
  const months = [
    'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'
  ];
  return '${weekdays[date.weekday - 1]}, ${date.day} ${months[date.month - 1]} ${date.year}';
}

// Toggle completion status of a task
Future<void> _toggleTaskCompletion(Task task, bool? newValue) async {
  final updatedTask = Task(
    id: task.id,
    title: task.title,
    description: task.description,
    date: task.date,
    isCompleted: newValue ?? !task.isCompleted,
  );

  // Update state immediately for instant UI feedback
  setState(() {
    final index = _tasks.indexWhere((t) => t.id == task.id);
    if (index != -1) {
      _tasks[index] = updatedTask;
    }
  });

  // Persist change to SharedPreferences
  await _storageService.updateTask(updatedTask);
}

// Navigate to Add Task Screen
Future<void> _navigateToAddTask() async {
  final result = await Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const AddTaskScreen()),
  );

  // If a new task was added, reload the task list
  if (result == true) {
    _loadTasks();
  }
}

// Navigate to Edit Task Screen
Future<void> _navigateToEditTask(Task task) async {
  final result = await Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => EditTaskScreen(task: task)),
  );

  // If the task was modified, reload the list
  if (result == true) {
    _loadTasks();
  }
}

// Show delete confirmation dialog
Future<void> _confirmDeleteTask(Task task) async {
  final shouldDelete = await showDialog<bool>(
    context: context,
    builder: (BuildContext dialogContext) {
      return AlertDialog(
        title: const Text('Delete this task?'),
        content: const Text('Are you sure you want to delete this task?'),
        actions: [
          // Cancel button
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Cancel'),
          ),
          // Confirm Delete button
          FilledButton(
            style: FilledButton.styleFrom(
              backgroundColor: Theme.of(context).colorScheme.error,
            ),
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Delete'),
          ),
        ],
      );
    },
  );

  if (shouldDelete == true) {
    // Remove task from state and storage
    setState(() {
      _tasks.removeWhere((t) => t.id == task.id);
    });

    await _storageService.deleteTask(task.id);

    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
```

```
      const SnackBar(
        content: Text('Task deleted successfully'),
        behavior: SnackBarBehavior.floating,
      ),
    );
  }
}

// Get appropriate empty message depending on filter
String _getEmptyMessage() {
  if (_tasks.isEmpty) {
    return 'No tasks yet';
  }
  switch (_selectedFilter) {
    case TaskFilter.pending:
      return 'No pending tasks';
    case TaskFilter.completed:
      return 'No completed tasks';
    case TaskFilter.all:
      return 'No tasks yet';
  }
}

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final displayedTasks = _filteredTasks;

  return Scaffold(
    appBar: AppBar(
      title: const Text(
        'Daily Task Manager',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      bottom: PreferredSize(
        preferredSize: const Size.fromHeight(28),
        child: Container(
          alignment: Alignment.centerLeft,
          padding: const EdgeInsets.only(left: 20, bottom: 8),
          child: Text(
            _formatTodayDate(DateTime.now()),
            style: theme.textTheme.bodyMedium?.copyWith(
              color: theme.colorScheme.onSurfaceVariant,
              fontWeight: FontWeight.w500,
            ),
          ),
        ),
      ),
    ),
    body: Column(
      children: [
        // Filter Chips / Segmented Control
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 16.0, vertical: 12.0),
          child: SegmentedButton<TaskFilter>(
            segments: const [
              ButtonSegment<TaskFilter>(
                value: TaskFilter.all,
                label: Text('All'),
                icon: Icon(Icons.list_alt),
              ),
              ButtonSegment<TaskFilter>(
                value: TaskFilter.pending,
                label: Text('Pending'),
                icon: Icon(Icons.pending_actions),
              ),
              ButtonSegment<TaskFilter>(
                value: TaskFilter.completed,
                label: Text('Completed'),
                icon: Icon(Icons.task_alt),
              ),
            ],
            selected: {_selectedFilter},
            onSelectionChanged: (Set<TaskFilter> newSelection) {
              setState(() {
                _selectedFilter = newSelection.first;
              });
            },
          ),
        ),

        // Main Task List or Empty State
        Expanded(
          child: _isLoading
              ? const Center(child: CircularProgressIndicator())
              : displayedTasks.isEmpty
                  ? Center(
                      child: Column(
                        mainAxisAlignment: MainAxisAlignment.center,
                        children: [
                          Icon(
                            _selectedFilter == TaskFilter.completed
                                ? Icons.task_alt
                                : Icons.assignment_outlined,
```

```
                            size: 64,
                            color: theme.colorScheme.outlineVariant,
                          ),
                          const SizedBox(height: 12),
                          Text(
                            _getEmptyMessage(),
                            style: theme.textTheme.titleMedium?.copyWith(
                              color: theme.colorScheme.onSurfaceVariant,
                            ),
                          ),
                        ],
                      ),
                    )
                  : ListView.builder(
                      padding: const EdgeInsets.only(top: 4, bottom: 80),
                      itemCount: displayedTasks.length,
                      itemBuilder: (context, index) {
                        final task = displayedTasks[index];
                        return TaskCard(
                          key: ValueKey(task.id),
                          task: task,
                          onToggleComplete: (bool? newValue) {
                            _toggleTaskCompletion(task, newValue);
                          },
                          onEdit: () {
                            _navigateToEditTask(task);
                          },
                          onDelete: () {
                            _confirmDeleteTask(task);
                          },
                        );
                      },
                    ),
            ),
          ],
        ),
        // Floating Action Button to Add New Task
        floatingActionButton: FloatingActionButton(
          onPressed: _navigateToAddTask,
          tooltip: 'Add Task',
          child: const Icon(Icons.add),
        ),
      );
  }
}
```

Listing 4.5: lib/screens/add_task_screen.dart

```
import 'package:flutter/material.dart';
import '../models/task.dart';
import '../services/storage_service.dart';

// Screen to create and add a new task
class AddTaskScreen extends StatefulWidget {
  const AddTaskScreen({super.key});

  @override
  State<AddTaskScreen> createState() => _AddTaskScreenState();
}

class _AddTaskScreenState extends State<AddTaskScreen> {
  final _formKey = GlobalKey<FormState>();
  final _titleController = TextEditingController();
  final _descriptionController = TextEditingController();
  final StorageService _storageService = StorageService();

  DateTime _selectedDate = DateTime.now();
  bool _isSaving = false;

  @override
  void dispose() {
    _titleController.dispose();
    _descriptionController.dispose();
    super.dispose();
  }

  // Format date helper: "28 Sep 2026"
  String _formatDate(DateTime date) {
    const months = [
      'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
      'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'
    ];
    return '${date.day} ${months[date.month - 1]} ${date.year}';
  }

  // Opens the Flutter Material 3 Date Picker
  Future<void> _pickDate() async {
    final DateTime? pickedDate = await showDatePicker(
      context: context,
      initialDate: _selectedDate,
      firstDate: DateTime(2020),
      lastDate: DateTime(2035),
    );
```

```
  if (pickedDate != null && pickedDate != _selectedDate) {
    setState(() {
      _selectedDate = pickedDate;
    });
  }
}

// Validates input and saves the task to SharedPreferences
Future<void> _saveTask() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  setState(() {
    _isSaving = true;
  });

  final newTask = Task(
    id: DateTime.now().millisecondsSinceEpoch.toString(),
    title: _titleController.text.trim(),
    description: _descriptionController.text.trim(),
    date: _selectedDate,
    isCompleted: false,
  );

  final success = await _storageService.addTask(newTask);

  if (!mounted) return;

  if (success) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Task added successfully!'),
        behavior: SnackBarBehavior.floating,
      ),
    );
    Navigator.pop(context, true); // Return true to indicate change
  } else {
    setState(() {
      _isSaving = false;
    });
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Failed to save task. Please try again.'),
        behavior: SnackBarBehavior.floating,
      ),
    );
  }
}

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);

  return Scaffold(
    appBar: AppBar(
      title: const Text('Add Task'),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(20.0),
      child: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            // Task Title Field (Required)
            TextFormField(
              controller: _titleController,
              textCapitalization: TextCapitalization.sentences,
              decoration: const InputDecoration(
                labelText: 'Task Title *',
                hintText: 'e.g., Complete Flutter Assignment',
                border: OutlineInputBorder(),
                prefixIcon: Icon(Icons.title),
              ),
              validator: (value) {
                if (value == null || value.trim().isEmpty) {
                  return 'Please enter a task title';
                }
                return null;
              },
            ),
            const SizedBox(height: 20),

            // Task Description Field (Optional)
            TextFormField(
              controller: _descriptionController,
              textCapitalization: TextCapitalization.sentences,
              maxLines: 3,
              decoration: const InputDecoration(
                labelText: 'Description (Optional)',
                hintText: 'Add details or notes about the task',
                border: OutlineInputBorder(),
```

```
                alignLabelWithHint: true,
                prefixIcon: Padding(
                  padding: EdgeInsets.only(bottom: 45),
                  child: Icon(Icons.notes),
                ),
              ),
            ),
            const SizedBox(height: 20),

            // Date Picker Selection
            InkWell(
              onTap: _pickDate,
              borderRadius: BorderRadius.circular(8),
              child: InputDecorator(
                decoration: const InputDecoration(
                  labelText: 'Task Date',
                  border: OutlineInputBorder(),
                  prefixIcon: Icon(Icons.calendar_month),
                  suffixIcon: Icon(Icons.arrow_drop_down),
                ),
                child: Text(
                  _formatDate(_selectedDate),
                  style: theme.textTheme.bodyLarge,
                ),
              ),
            ),
            const SizedBox(height: 32),

            // Save Button
            FilledButton.icon(
              onPressed: _isSaving ? null : _saveTask,
              icon: _isSaving
                  ? const SizedBox(
                      width: 18,
                      height: 18,
                      child: CircularProgressIndicator(
                        strokeWidth: 2,
                        color: Colors.white,
                      ),
                    )
                  : const Icon(Icons.check),
              label: Text(
                _isSaving ? 'Saving...' : 'Save Task',
                style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
              ),
              style: FilledButton.styleFrom(
                padding: const EdgeInsets.symmetric(vertical: 14),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
              ),
            ),
          ],
        ),
      ),
    ),
  );
  }
}
```

Listing 4.6: test/unit_and_flow_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:daily_task_manager/models/task.dart';
import 'package:daily_task_manager/services/storage_service.dart';
import 'package:daily_task_manager/screens/home_screen.dart';

void main() {
  group('Task Model & StorageService Tests', () {
    setUp(() {
      SharedPreferences.setMockInitialValues({});
    });

    test('Task converts to JSON and back from JSON correctly', () {
      final now = DateTime(2026, 9, 28, 12, 0, 0);
      final task = Task(
        id: '123',
        title: 'Complete Flutter Assignment',
        description: 'Finish the Daily Task Manager',
        date: now,
        isCompleted: false,
      );

      final json = task.toJson();
      expect(json['id'], '123');
      expect(json['title'], 'Complete Flutter Assignment');
      expect(json['description'], 'Finish the Daily Task Manager');
      expect(json['isCompleted'], false);

      final reconstructed = Task.fromJson(json);
      expect(reconstructed.id, task.id);
```

```
    expect(reconstructed.title, task.title);
    expect(reconstructed.description, task.description);
    expect(reconstructed.date, task.date);
    expect(reconstructed.isCompleted, task.isCompleted);
  });

  test('StorageService saves, loads, updates, and deletes tasks', () async {
    final storage = StorageService();

    // Initially empty
    final initialTasks = await storage.loadTasks();
    expect(initialTasks, isEmpty);

    // Add a task
    final task1 = Task(
      id: '1',
      title: 'Task 1',
      description: 'First Description',
      date: DateTime(2026, 9, 28),
    );
    await storage.addTask(task1);

    var tasks = await storage.loadTasks();
    expect(tasks.length, 1);
    expect(tasks.first.title, 'Task 1');

    // Update task
    task1.isCompleted = true;
    await storage.updateTask(task1);

    tasks = await storage.loadTasks();
    expect(tasks.first.isCompleted, true);

    // Delete task
    await storage.deleteTask('1');
    tasks = await storage.loadTasks();
    expect(tasks, isEmpty);
  });
});

group('HomeScreen Interactive Tests', () {
  setUp(() {
    SharedPreferences.setMockInitialValues({});
  });

  testWidgets('Full Task Flow: Add, Toggle, Filter, and Delete', (WidgetTester tester) async {
    final storage = StorageService();
    // Pre-populate with two tasks: one pending, one completed
    await storage.addTask(
      Task(
        id: 'task-done',
        title: 'Completed Task',
        description: 'This is completed',
        date: DateTime(2026, 9, 28),
        isCompleted: true,
      ),
    );
    await storage.addTask(
      Task(
        id: 'task-pending',
        title: 'Pending Task',
        description: 'This is pending',
        date: DateTime(2026, 9, 28),
        isCompleted: false,
      ),
    );

    await tester.pumpWidget(const MaterialApp(home: HomeScreen()));
    await tester.pumpAndSettle();

    // Verify both tasks are visible in "All" view
    expect(find.text('Pending Task'), findsOneWidget);
    expect(find.text('Completed Task'), findsOneWidget);

    // Switch filter to "Pending"
    await tester.tap(find.text('Pending'));
    await tester.pumpAndSettle();
    expect(find.text('Pending Task'), findsOneWidget);
    expect(find.text('Completed Task'), findsNothing);

    // Switch filter to "Completed"
    await tester.tap(find.text('Completed'));
    await tester.pumpAndSettle();
    expect(find.text('Pending Task'), findsNothing);
    expect(find.text('Completed Task'), findsOneWidget);

    // Switch back to "All"
    await tester.tap(find.text('All'));
    await tester.pumpAndSettle();

    // Test Delete Dialog: Tap delete icon on Pending Task
    final deleteIcons = find.byIcon(Icons.delete_outline);
    await tester.tap(deleteIcons.first);
    await tester.pumpAndSettle();
```

```
      // Dialog should appear
      expect(find.text('Delete this task?'), findsOneWidget);
      expect(find.text('Are you sure you want to delete this task?'), findsOneWidget);

      // Tap Delete in dialog
      await tester.tap(find.widgetWithText(FilledButton, 'Delete'));
      await tester.pumpAndSettle();

      // One task should remain
      expect(find.byType(Card), findsNWidgets(1));
    });
  });
}
```

CHAPTER 5: Results

Output of the Daily Task Manager

5.1 Screenshots or examples of the application in action: When the app is opened for the first time it shows today's date, the All, Pending and Completed filter and the message No tasks yet. The plus button opens the Add Task screen. After saving, the task appears at the top of the list, and finished tasks are shown with a strike-through and a lighter card. Figures 5.1 and 5.2 show the main screens on a phone and Table 5.1 lists the messages the app gives for typical actions.

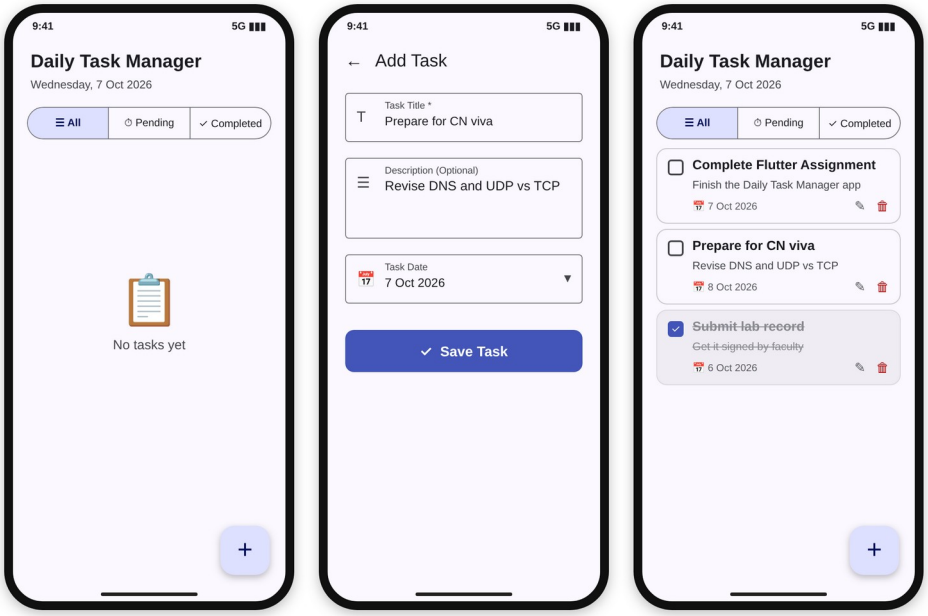


Figure 5.1: Empty home screen, Add Task screen and home screen with tasks

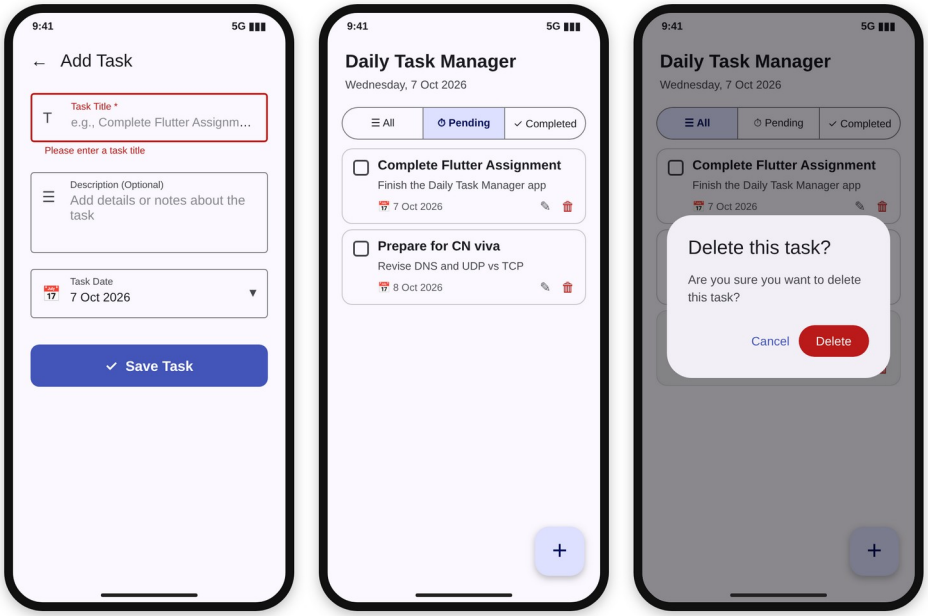


Figure 5.2: Title validation error, Pending filter and delete confirmation dialog

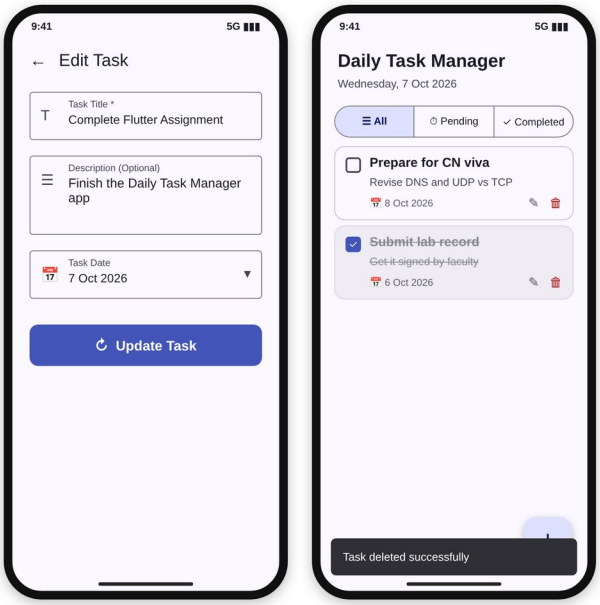


Figure 5.3: Edit Task screen and the message shown after deleting a task

Table 5.1: Behaviour of the application

Action	Response of the application
Open the app with no saved tasks	Message: No tasks yet
Select Pending or Completed with no matching task	Message: No pending tasks or No completed tasks
Save with an empty title	Field error: Please enter a task title
Save a valid task	Snack bar: Task added successfully! and the task appears on the home screen
Tap the checkbox	Task becomes completed, title is struck through, change is saved
Update a task	Snack bar: Task updated successfully!
Tap the delete icon	Dialog: Delete this task? with Cancel and Delete
Confirm delete	Task removed from the list and storage, snack bar: Task deleted successfully

5.2 Discussion of the results: The project contains four automated tests. One unit test checks that a Task converts to JSON and back without losing any field. A second unit test checks that StorageService can add, load, update and delete a task. A widget test fills storage with one pending and one completed task, checks the All, Pending and Completed filters, opens the delete dialog and confirms the deletion. A second widget test checks that the app starts with the title, the empty state and the three filter buttons. The tests use mock SharedPreferences values so they never touch real data. Saved tasks stay available after the app is restarted because they are stored on the device.

CHAPTER 6: Conclusion and Future Scope

Summary of the project

6.1 Recap of the project's objectives and achievements: The project set out to build a complete but simple task manager in Flutter. The app supports creating, viewing, editing, completing, filtering and deleting tasks, stores them permanently on the device, validates

input and gives clear feedback. The code is divided into model, service, widget and screen files and is covered by automated tests.

6.2 Discussion of the project's limitations: The app keeps data only on one device and has no backup or synchronisation. It has no reminders or notifications, no search or sorting options, and no categories or priorities. The whole task list is read and written each time, which suits a small list but not thousands of tasks, and only a light theme is defined.

Future scope and enhancements

6.3 Ideas for future improvements and enhancements: Local notifications can remind the user before a task is due. A database such as SQLite or Hive, or a cloud service such as Firebase, can replace SharedPreferences for larger data and synchronisation. Search, sorting, priorities, categories, a dark theme and a calendar view would make the application more complete.

References

[1] Flutter documentation, https://docs.flutter.dev

[2] Dart language tour, https://dart.dev/language

[3] shared_preferences package, https://pub.dev/packages/shared_preferences

[4] Flutter cookbook: Forms and validation, https://docs.flutter.dev/cookbook/forms

[5] Flutter testing overview, https://docs.flutter.dev/testing/overview

[6] Material Design 3, https://m3.material.io